

Tome 2 — Le playbook de la compétition

Pour les 4 membres. C'est **le** document de référence du week-end : règles, critères, organisation, qui fait quoi, quand et comment. En cas de désaccord pendant la compétition, on applique ce qui est écrit ici, et on en rediscute après.

0. L'essentiel en 10 lignes

1. Le sujet tombe **samedi 3 octobre à 9 h**. Fin du développement **dimanche 4 octobre à 9 h**.
2. ~7 fonctionnalités **de base** au départ, puis de **nouvelles fonctionnalités toutes les 2-3 h**. Il est prévu qu'on ne puisse **pas tout faire** : **choisir** fait partie de la note.
3. Le jury **teste lui-même** l'application en ligne, les jours suivants, sans nous. Il vérifie chaque fonctionnalité déclarée et **attaque la sécurité**.
4. Ce qui n'est pas **en ligne sur Hodi** n'existe pas. Ce qui n'est pas **dans le récap** ne sera pas cherché.
5. Mieux vaut **8 fonctionnalités finies, belles et sûres** que 15 à moitié.
6. **Socle en ligne à H+6**, déploiement toutes les **2-3 h**, **gel à H+21** (6 h du matin).
7. Une personne décide (**Randy**, ou **Judicaël** quand Randy dort). Triage d'une annonce : **10 minutes maximum**.
8. Rien n'est « terminé » tant que **Njaraniaina** ne l'a pas testé **en ligne** avec les comptes jury.
9. On dort **par roulement** : jamais le chef et l'adjoint en même temps.
10. Livrables (URL, comptes, récap, vidéo) : **dimanche 11 h → lundi 11 h**. Objectif : **tout envoyé dimanche 18 h**.

1. Le règlement, à la lettre

Source : page officielle Notion « 24h by Webcup » (sections 2 à 10, relues le 30/09) et annonces Discord. **Les citations sont exactes.**

1.1 Calendrier

Moment	Ce qui se passe
J-7 (depuis le 26/09)	Serveur ouvert : « installation des outils, frameworks et dépendances », « tests de déploiement ».
Sam. 3/10, 9 h	Lancement : concept, thème, fonctionnalités de base , consignes, règles.
Sam. 9 h → Dim. 9 h	Développement. « De nouvelles fonctionnalités sont annoncées par l'organisation selon une fréquence définie à l'avance. »
Dim. 4/10, 9 h	Fin du développement.
Dim. 11 h → Lun. 5/10, 11 h	Remise des livrables (formulaire du dashboard Webcup).
Lun. 15 h → sam. suivant	Évaluation par le jury.
Après	Remise des prix (podium 1er / 2e / 3e + distinctions).

⚠ À confirmer sur Discord (voir §11) : fuseau horaire exact, possibilité de pousser du code entre 9 h et 11 h (**par défaut : non**), application en ligne pendant toute l'évaluation (**par défaut : oui, on ne touche plus au serveur**).

1.2 Ce que dit le règlement sur le développement

- « Le volume total de fonctionnalités envisagé par l'organisation est volontairement ambitieux, afin qu'il soit difficile, voire impossible, de tout implémenter dans les 24 heures. »
- Pour chaque annonce, les équipes décident : « l'implémenter immédiatement », « la traiter plus tard », ou « faire l'impasse ».
- Catégories possibles des annonces : « fonctionnalités métier, améliorations d'interface, intégrations techniques, exploitation de données et fonctionnalités liées à la sécurité applicative ».
- Équilibre demandé entre « l'avancement fonctionnel », « la qualité technique », « la stabilité » et « la qualité de l'interface ».
- « L'objectif n'est pas uniquement d'ajouter le plus grand nombre possible de fonctionnalités, mais de produire une application web cohérente, fonctionnelle et maîtrisée. »

1.3 Sécurité (section 6)

Familles testées : « l'authentification ; le contrôle d'accès ; la validation des entrées utilisateur ; la protection de certains endpoints ; l'exposition involontaire de données ; des formes simples de brute force ou de mauvaise gestion des rôles ». Et : «

certaines demandes de fonctionnalités annoncées pendant les 24 heures pourront également intégrer une dimension sécurité ».

1.4 Livrables (section 7)

Au minimum :

1. « l'URL de l'application web » ;
2. « les accès nécessaires au jury » (suffisants pour vérifier toutes les parties, y compris celles qui demandent une connexion) ;
3. « un récapitulatif des fonctionnalités implémentées » ;
4. « une courte vidéo de démonstration ».

« Seuls les éléments effectivement remis à la clôture du concours pourront être pris en compte. » Nous ajoutons le **lien GitHub** et un **README** (demandés dans le formulaire / utiles au jury pour la qualité technique).

1.5 Grille d'évaluation (section 8)

Dimension	Ce que le jury regarde (texte officiel)	Ce qui le prouve chez nous
Fonctionnalités	« présence, niveau de fonctionnement, intégration réelle » ; « pertinence, niveau de finition, bon fonctionnement »	Récap exact, chaque ligne testée en ligne, données de démo
Qualité technique	« cohérence de la structure », « logique de développement », « robustesse », « qualité de l'intégration », « exploitation des données ou des API », « sécurité applicative »	Convention, Politiques, tests, CI verte, README, pas d'erreur 500
Design et UX	« lisibilité, ergonomie, navigation, qualité graphique, expérience utilisateur »	Identité visuelle cohérente, mobile, 4 états par écran
Cohérence d'ensemble	« équilibre entre ambition et exécution », « logique fonctionnelle », « qualité globale du rendu »	Un fil conducteur, un menu clair, pas de fonctionnalité orpheline

Distinctions possibles : qualité graphique, UX, qualité technique, fonctionnalité particulièrement réussie, originalité / cohérence.

1.6 Outils et IA (section 10 + page IA)

- Technologies libres ; « l'usage des outils d'intelligence artificielle est également autorisé ».
- Mais « la seule utilisation d'outils automatisés ne suffit pas » : ce qui compte, ce sont les **choix**, la **structure**, l'**intégration**, la **maîtrise**.
- La préparation J-7 est « exclusivement consacrée à la préparation technique » → notre socle générique (Laravel, auth, admin, générateur, déploiement) est dans l'esprit. **Aucun code métier lié au sujet avant 9 h samedi**. L'exemple *Signalement* est retiré vendredi.
- OpenRouter (IA dans l'application) : modèles : free , **20 requêtes/minute**, **50/jour** sur un compte neuf, **1000/jour** après un premier crédit. « Créer plusieurs comptes ne sert à rien. » Clé **jamais** dans le code public.

2. L'équipe et les rôles

Qui	Rôle	En une phrase
Randy	Chef d'équipe · intégration · déploiement	Décide les priorités, merge, déploie, pilote Claude.
Manakasina Judicaël	Adjoint · touche-à-tout	Prend les fonctionnalités difficiles, la sécurité, l'admin ; remplace Randy quand il dort.
Tsoa (Voa-hary)	Exécuteur	Produit les fonctionnalités en continu (générateur + adaptation + interface).
Njaraniaina	Scrum master · testeuse	Tient le board et le rythme, teste tout en ligne, garde le récap.
Claude	5 ^e membre (sessions cloud)	Analyse le sujet, code en parallèle sur des branches, audite la sécurité, prépare le récap.

Règles de fonctionnement

- **Une seule voix décide** : Randy (ou Judicaël s'il est en service). On propose, on argumente 2 minutes, puis on applique.
- **Personne ne reste bloqué plus de 20 minutes** : on le dit au point suivant ou tout de suite sur le vocal.
- **Chacun a toujours une carte « En cours »**, et une seule.
- **Personne ne merge sa propre PR** sauf Randy/Judicaël sur des corrections urgentes.
- **On ne pousse jamais sur main directement**, sauf le commit de gel et les correctifs d'urgence du chef.

3. La convention, à la lettre

Le détail est dans `CONVENTION.md` (repo). Les points qui font perdre ou gagner des points :

Sécurité (non négociable)

1. Toute route métier dans le groupe `auth` de `routes/features.php` (sauf pages publiques décidées).
2. Chaque méthode publique d'un composant Livewire appelle `$this->authorize(...)`.
3. `user_id`, `role`, `statut` réservés : **jamais** dans `#[Fillable]`, assignés dans le code.
4. Validation serveur (`rules()`) sur **chaque** champ, avec `Rule::in(...)` pour les listes.
5. Affichage avec `{{ }}` uniquement. Jamais `{!! !!}` sur une donnée utilisateur.
6. Uploads : `image|mimes:jpg,jpeg,png,webp|max:2048`.
7. Secrets dans `.env` uniquement. `APP_DEBUG=false` en production.
8. Tout enregistrement modifiable est `#[Locked]` dans le composant.

Code

- Nommage métier **en français**, comme le sujet (`Signalement`, `titre`, `niveau`).
- Une entité = `make:feature` d'abord, adaptation ensuite. On n'écrit pas un CRUD à la main.
- Typage des paramètres et retours ; pas de `declare(strict_types=1)`.
- `vendor/bin/pint` puis `php artisan test` **avant chaque push**.

Git

- Branche `feat/<sujet-court>` ou `fix/<sujet-court>` depuis `main` à jour.
- Commits en français : `feat: carte des points de regroupement`.
- PR : titre clair, Refs `#12` (**jamais** `Closes #12`), et la section « Comment tester ».

Interface

- Flux + Tailwind, mobile d'abord, textes en français cohérents avec le thème.
- Chaque écran gère : **vide, chargement, erreur, succès**.
- Chaque entité a des **données de démo crédibles**.

Définition de « terminé » (toutes les cases, sinon ce n'est pas terminé)

- ☐ Mergé dans `main`, CI verte
- ☐ Déployé sur Hodi
- ☐ Testé en ligne par Njaraniaina avec le compte jury **user**, et **admin** si concerné
- ☐ Un autre utilisateur ne peut ni voir ni modifier ce qui ne le concerne pas
- ☐ Utilisable sur téléphone
- ☐ Ligne ajoutée au récap → issue fermée

4. Le workflow d'une fonctionnalité

Annonce → Triage (10 min) → Issue + priorité → Branche → Code + tests
→ PR (CI) → Merge → Déploiement → Recette en ligne → Récap → Fermée

Étape	Qui	Comment	Durée
1. Triage	Randy (Judicaël)	/triage <texte> dans Claude Code ; décider P0/P1/P2/non	≤ 10 min
2. Issue	Njaraniaina	Modèle « Fonctionnalité » : texte exact de l'annonce, critères d'acceptation, priorité, label, responsable	3 min
3. Démarrer	Dev	Carte → « En cours » ; <code>git checkout main && git pull && git checkout -b feat/xxx</code>	1 min
4. Coder	Dev (+ Claude)	<code>make:feature</code> puis adaptation, ou /fonctionnalite <description>	30-90 min
5. Vérifier en local	Dev	<code>pint</code> , <code>php artisan test</code> , test à la main (compte user + compte « autre »)	5 min
6. PR	Dev	<code>git push -u origin feat/xxx</code> ; PR avec Refs #n et « Comment tester »	3 min
7. Merge	Randy (Judicaël)	CI verte + relecture 2 min (auth, authorize, test 403) → merge	5 min
8. Déployer	Randy (Judicaël)	<code>bash ~/webcup-2026/deploy.sh</code> (regrouper plusieurs merges)	5 min
9. Recette	Njaraniaina	Checklist §7 sur l'URL en ligne, navigation privée, téléphone	5-10 min
10. Récap	Njaraniaina	Ligne dans docs/recap.md → ferme l'issue	2 min

Si la recette échoue : Njaraniaina commente l'issue (ce qui ne marche pas, capture), la remet en « En cours » et prévient le dev. Pas de débat : on corrige ou on décide de retirer.

5. Le board GitHub

Projet #17 « Webcup 2026 ». Une carte = une issue.

Colonne	Qui y met la carte	Quand
À faire	Njaraniaina (création de l'issue)	Après le triage
En cours	Le dev lui-même	Quand il commence (et une seule carte par personne)
À tester sur Hodi	Automatique	Quand la PR est mergée (Refs #n) — sinon Randy la déplace
Fait & dans le récap	Njaraniaina	Après recette en ligne réussie + ligne dans le récap (elle ferme l'issue)

Champs et labels

- **Priorité** : P0 base obligatoire · P1 forte valeur jury, effort raisonnable · P2 si le temps le permet.
- **Labels** : base (fonctionnalité de départ) · progressive (annoncée pendant le concours) · sécu (dimension sécurité) · prépa (préparation avant le jour J) · bug .
- **Titre** : court et métier, préfixé du numéro d'annonce pour les progressives : [A3] Export CSV des signalements .
- **Responsable** : une seule personne.

Lecture du board en 10 secondes (au point de 2 h) : trop de cartes « À tester » = déployer ; une carte « En cours » depuis plus de 2 h = bloquée, on en parle.

6. Le triage d'une annonce

Décidé par Randy (Judicaël s'il est de service), **10 minutes maximum**, annoncé en une phrase sur le vocal.

Question	Si oui
Fonctionnalité de base ?	P0 , toujours
Déjà couverte par ce qu'on a (rôles, admin, upload, filtres, auth) ?	P1 tout de suite : points presque gratuits
Dimension sécurité ?	P1 : critère explicite du jury
Moins d'1 h et visible par le jury ?	P1
Plus de 1 h 30 et touche le cœur de l'application ?	P2 ou non
Risque de casser ce qui marche, ou dépend d'un service externe fragile ?	P2 , seulement si le reste est stable
Après H+19 ?	Non , sauf si < 30 min et sans migration

Exemples (sujets d'entraînement)

- *SafeZone* — « Permettre aux utilisateurs de signaler un incident avec photo et position » → base, P0, `make:feature Incident` --
`fields="titre:string,description:text,niveau:enum(faible/moyen/critique),photo:image?,latitude:decimal?,longitude:decimal?"` , Tsoa.
- *SafeZone* — « Limiter le nombre de signalements par utilisateur et par heure » → sécu, P1, 20 min (`RateLimiter`), Judicaël.
- *FutureCity* — « Proposer un assistant IA qui résume les propositions citoyennes » → progressive, P1 si le socle est stable (service IA + cache + limitation), sinon P2. Claude en session cloud.
- *FutureCity* — « Application mobile native » → **non** (hors périmètre web, énorme).

7. La recette en ligne (Njaraniaina)

Pour **chaque** fonctionnalité, sur l'URL en ligne, en navigation privée :

- ☐ Le parcours décrit dans l'issue fonctionne avec le compte **jury user**
- ☐ Les parties admin fonctionnent avec le compte **jury admin**
- ☐ Avec un **deuxième compte user** : impossible de voir/modifier les données du premier (changer l'ID dans l'URL → 403 ou 404)
- ☐ Formulaire : champs vides → messages en français ; texte `<script>alert(1)</script>` → affiché comme du texte
- ☐ Liste vide → message utile ; données de démo présentes
- ☐ Largeur téléphone : lisible, boutons cliquables, pas de débordement
- ☐ Aucune erreur 500, aucune page anglaise oubliée

Les **5 attaques du jury** (à refaire sur toute l'app à H+21) : changer l'ID dans l'URL ; appeler une action Livewire d'un autre ; ajouter `role=admin` ou `user_id=1` à un formulaire ; ouvrir `/admin` en user ; injecter du HTML.

8. Le déroulé des 24 heures

8.1 Avant le départ (samedi)

Heure	Quoi	Qui
7 h 30	Arrivée, installation, Herd lancé, <code>git pull</code> , <code>composer run dev</code> OK	Tous
8 h 00	Vérification : site en ligne, <code>/admin</code> OK, CI verte, sessions cloud prêtes, OpenRouter OK	Randy
8 h 30	Vocal ouvert, board vide prêt, café	Njaraniaina
8 h 55	Discord et dashboard Webcup ouverts	Tous

8.2 H0 → H+1 : le lancement

Heure	Quoi	Qui
9 h 00	Chacun lit le sujet (10 min, en silence)	Tous
9 h 00	Le sujet complet est collé à Claude : modèle de données, backlog P0/P1/P2, direction visuelle, commandes <code>make:feature</code>	Randy
9 h 10	Réunion de 15 min : valider les entités et la répartition des fonctionnalités de base	Tous, animée par Njaraniaina
9 h 25	Issues créées, board rempli	Njaraniaina
9 h 25	Génération des entités principales, commit, push sur <code>main</code>	Randy
9 h 40	Chacun part de <code>main</code> à jour sur sa carte	Tous

8.3 Le reste de la journée et la nuit

Heure (H+)	Randy	Judicaël	Tsoa	Njaraniaina
9 h 40 → 15 h (→ H+6)	Intègre, merge, 1 ^{er} déploiement vers 13 h, pilote Claude	Base difficile (relations, rôles, sécu)	Base (entités, pages, identité visuelle)	Board, issues, recette, comptes jury, récap
15 h (H+6)	Socle en ligne obligatoire			Recette complète
15 h → 21 h	Triage, merges, déploiement toutes les 2-3 h	Progressives difficiles	Progressives	Recette après chaque déploiement
21 h → 22 h 30	Chef	Progressives	Dort (21 h → 0 h 30)	Recette
22 h 30 → 0 h 30	Chef	Dort (22 h 30 → 2 h)	<i>dort</i>	Recette
0 h 30 → 2 h	Chef	<i>dort</i>	Progressives	Dort (0 h 30 → 4 h)
2 h → 4 h	Dort (2 h → 5 h 30)	Chef par intérim	Progressives	<i>dort</i>
4 h → 6 h	<i>dort</i> , reprend à 5 h 30	Chef par intérim, rend la main à 5 h 30	Progressives	Recette
6 h (H+21)	GEL : plus aucune nouvelle fonctionnalité			
6 h → 8 h	/audit-sécu, corrections	5 attaques du jury, corrections	Mobile, finitions visuelles, textes	Recette complète de tout le récap
8 h 00	README final commité			Récap finalisé
8 h 30	Dernier déploiement	Test complet en ligne	Test mobile	Test comptes jury
8 h 45	Tag v1.0-rendu sur main ; plus aucun push			
9 h 00	Fin			

Points de 5 minutes (animés par Njaraniaina) : 11 h, 13 h, 15 h, 17 h, 19 h, 21 h, 23 h, 1 h, 3 h, 5 h, 7 h. Chacun : fait / en cours / bloqué. Puis lecture du board.

Sommeil : les créneaux ci-dessus sont fixes. On ne « saute » pas son sommeil : une personne épuisée à 5 h casse plus qu'elle ne construit.

8.4 Dimanche après 9 h : les livrables

Heure	Quoi	Qui
9 h → 11 h 30	Dormir (ou au moins se reposer)	Tous
11 h 30 → 13 h	/recap-jury → récap final relu	Njaraniaina + Randy
13 h → 15 h	Tournage de la vidéo sur l'application en ligne (script §9)	Randy (voix) + Njaraniaina (réalisation)
15 h → 16 h	Montage simple, export MP4, mise en ligne (YouTube non répertorié ou Drive public)	Njaraniaina
16 h → 17 h	Formulaire Webcup rempli, relu par Judicaël	Randy
18 h	Tout est envoyé (marge jusqu'à lundi 11 h)	Randy

9. Les livrables en détail

README (commité avant 8 h dimanche) : présentation (3 lignes), URL, comptes jury, fonctionnalités (renvoi au récap), architecture (stack, dossiers), sécurité (liste des protections), installation locale, équipe, section « Outils préparés avant la compétition » (socle générique, générateur).

Récap des fonctionnalités (docs/recap.md , tenu par Njaraniaina pendant le concours) :

#	Fonctionnalité	Type	Où la voir	Compte	Comment tester
B1	Page d'accueil	base	/	aucun	Ouvrir le site
A3	Export CSV des signalements	progressive	/admin/signalements	admin	Bouton « Exporter »

Uniquement ce qui est dans la colonne « Fait & dans le récap ». Les fonctionnalités de sécurité y sont aussi (limite de connexion, rôles, 2FA...).

Vidéo (3 min maximum), script type :

1. 0:00-0:20 — Le problème et l'application en une phrase, page d'accueil.
2. 0:20-2:10 — Parcours utilisateur réel sur 4-5 fonctionnalités clés (compte user).
3. 2:10-2:40 — Espace admin : gestion, modération, statistiques.
4. 2:40-3:00 — Sécurité et qualité en 3 points, écran de fin avec l'URL.

Tournée sur l'URL **en ligne**, pas en local. Outil : OBS Studio (testé jeudi). Micro correct, pas de musique forte.

Comptes jury : `jury@...` (user) et `jury-admin@...` (admin), mots de passe uniques ≥ 12 caractères, notés dans le README et le formulaire. Créés dès que le socle est en ligne, jamais supprimés.

10. Le plan d'urgence

Situation	Réflexe
Production cassée (500)	<code>tail -n 60 storage/logs/laravel.log</code> ; <code>php84 artisan optimize:clear && php84 artisan optimize</code> . Si une fonctionnalité en est la cause : <code>git revert <sha></code> en local, push, redéploiement. Pas de <code>reset --hard</code> sur le serveur.
Page sans style	<code>deploy.sh</code> a raté <code>npm run build</code> : relancer, lire l'erreur. Ctrl+F5.
Migration qui échoue en ligne	Différence SQLite/MariaDB. Corriger en local, nouvelle migration, redéploier. Ne jamais modifier une migration déjà passée.
Conflit Git	<code>routes/features.php</code> , <code>sidebar</code> , <code>seeder</code> : garder les deux blocs. Ailleurs : l'auteur de la PR résout, aidé de Judicaël.
Un membre bloqué > 20 min	Il le dit ; Judicaël ou Claude aide ; si toujours bloqué à 40 min, Randy change la priorité.
Quota Claude épuisé	Passer sur Sonnet ; le générateur et les guides suffisent pour un CRUD ; un seul pilote Claude (Randy).
OpenRouter en panne ou quota atteint	Changer <code>OPENROUTER_MODEL</code> dans <code>.env</code> du serveur, <code>php84 artisan optimize</code> . Le service affiche un repli propre, jamais une erreur.
API de l'orga en panne	Données en cache / en base, message clair.
Coupure Internet / électricité	Partage de connexion 4G (préparé), batterie externe, le serveur continue de tourner.
Retard sur le planning	À H+18, tout ce qui n'est pas commencé passe en « non ». Le gel à H+21 ne bouge pas.

11. Préparation : le backlog jusqu'à samedi

Toutes ces tâches sont des issues du board (label `prépa`), créées par `scripts/creer-backlog.ps1`.

Mercredi 30/09

Tâche	Qui	Fin quand
Question Discord : code générique préparé autorisé ? + fuseau horaire, push après 9 h, app en ligne pendant l'évaluation	Randy	Réponse notée dans l'issue
Page d'accueil + identité visuelle (session cloud)	Randy	PR mergée, déployée
Tester le nouveau <code>deploy.sh</code> (sauvegarde de la base, contrôle de santé, journal)	Randy	Fichier dans <code>~/backups</code> et « /up - > 200 OK »
Cron <code>schedule:run</code> installé et vérifié sur cPanel	Randy	Tâche test exécutée en ligne
<code>APP_LOCALE=fr</code> en production + déploiement	Randy	Site en français en ligne
Guides, playbook, installation (Claude)	Randy	PDF dans <code>docs/guides/</code>
Réunion d'équipe 21 h (30 min)	Njaraniaina	Décisions notées
Installation du projet chez chacun (<code>INSTALLATION.md</code>)	Judicaël, Tsoa, Njaraniaina	Connexion + /signalements OK en local
Premier cycle complet : une petite PR chacun, mergée et déployée	Judicaël, Tsoa	PR mergée

Jeudi 1/10

Tâche	Qui	Fin quand
Lecture tome 1 (Laravel) + son guide de rôle	Tous	Exercices faits
Générateur : options <code>--belongs-to</code> , <code>--statut</code> , <code>--public</code> , ressource Filament	Judicaël (+ Claude)	Tests verts, doc à jour
Compte OpenRouter, clé en <code>.env</code> serveur, modèle de secours noté	Judicaël	Appel test réussi en ligne
Envoi d'e-mail testé sur le serveur	Judicaël	Mail reçu
Lighthouse mobile ≥ 90 sur accueil et une liste	Tsoa	Scores notés
Comptes jury créés en production	Randy	Connexion OK
Modèles README, recap, script vidéo prêts	Njaraniaina	Fichiers dans <code>docs/</code>
OBS installé, enregistrement test de 30 s	Njaraniaina	Vidéo test lisible
Simulation 21 h (sujet d'entraînement, 2 h) + rétro 20 min	Tous (animée par Njaraniaina)	Leçons notées
Gel de la préparation à minuit	Randy	Plus de nouveau module

Vendredi 2/10

Tâche	Qui	Fin quand
Retrait de l'exemple Signalement (+ migration de suppression)	Randy	Déployé, repo sans entité métier
PDF des guides régénérés	Randy (Claude)	Dans <code>docs/guides/</code>
Logistique : lieu, repas, eau, café, rallonges, 4G, chargeurs	Njaraniaina	Liste cochée
Test à vide : <code>deploy.sh</code> , site, <code>/admin</code> , CI	Randy	Tout vert
Dormir tôt	Tous	22 h

12. Annexes

12.1 Prompts Claude prêts à coller

Au lancement (9 h)

Voici le sujet complet du 24h by Webcup (texte ci-dessous). Nous avons un socle Laravel 13 / Livewire 4 / Flux / Filament décrit dans `CLAUDE.md`, et le générateur `make:feature`. Donne :

- le modèle de données (entités, champs, relations) ;
- les commandes `make:feature` exactes pour les entités principales ;
- le backlog des fonctionnalités de base en P0/P1/P2, avec estimation et responsable proposé (Judicaël = difficile/sécu, Tsoa = pages/CRUD, Claude = parallèle) ;
- une direction visuelle (couleur d'accent Tailwind, ton des textes, icônes) ;
- les pièges de sécurité propres à ce sujet.

[SUJET]

Session cloud pour une fonctionnalité

Lis `CLAUDE.md`. Fonctionnalité `#<n>` : `<texte exact de l'annonce>`. Branche `feat/<nom>`. Respecte `CONVENTION.md` (authorize partout, validation, champs réservés hors Fillable, 4 états d'écran, français). Écris les tests (dont « autre utilisateur → 403 »). Pint, tests, puis PR avec Refs `#<n>` et une section « Comment tester en ligne ».

Commandes Claude Code : `/triage`, `/fonctionnalite`, `/audit-secu`, `/recap-jury`.

12.2 Commandes du week-end

```
# Local
git checkout main && git pull && git checkout -b feat/xxx
php artisan make:feature Nom --fields="..." --label="..." --plural="..." --icon=...
php artisan migrate
vendor/bin/pint && php artisan test
git push -u origin feat/xxx

# Serveur
bash ~/webcup-2026/deploy.sh
tail -n 60 ~/webcup-2026/storage/logs/laravel.log
```

12.3 Ce qu'on ne fait jamais

`migrate:fresh` ou `db:seed` en production · `APP_DEBUG=true` en ligne · une clé dans le code · merger sans CI verte · déployer sans tester ensuite · démarrer une fonctionnalité après le gel · déclarer une fonctionnalité non testée en ligne · pousser après 8 h 45 dimanche.